Software Testing Process:-

- **Step-1: Assess Development Plan and Status -** This initiative may be prerequisite to putting together Verification, Validation, and Testing Plan wont to evaluate implemented software solution. During this step, testers challenge completeness and correctness of event plan. Based on extensiveness and completeness of Project Plan testers can estimate quantity of resources they're going to got to test implemented software solution.

- **Step-2: Develop the Test Plan -** Forming plan for testing will follow an equivalent pattern as any software planning process. The structure of all plans should be an equivalent, but content will vary supported degree of risk testers perceive as related to software being developed.

- **Step-3: Test Software Requirements -** Incomplete, inaccurate, or inconsistent requirements cause most software failures. The inability to get requirement right during requirements gathering phase can also increase cost of implementation significantly. Testers, through verification, must determine that requirements are accurate, complete, and they do not conflict with another.

- **Step-4: Test Software Design -** This step tests both external and internal design primarily through verification techniques. The testers are concerned that planning will achieve objectives of wants, also because design being effective and efficient on designated hardware.

- **Step-5: Build Phase Testing -** The method chosen to build software from internal design document will determine type and extensiveness of testers needed. As the construction becomes more automated, less testing are going to be required during this phase. However, if software is made using waterfall process, it's subject to error and will be verified. Experience has shown that it's significantly cheaper to spot defects during development phase, than through dynamic testing during test execution step.

- **Step-6: Execute and Record Result -** This involves testing of code during dynamic state. The approach, methods, and tools laid out in test plan are going to be wont to validate that executable code actually meets stated software requirements, and therefore the structural specifications of design.

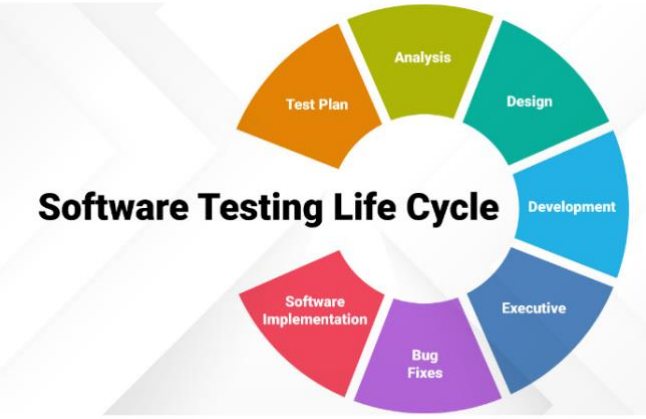
- **Step-7: Acceptance Test -** Acceptance testing enables users to gauge applicability and usefulness of software in performing their day-to-day job functions. This tests what user believes software should perform, as against what documented requirements state software should perform.

- **Step-8: Report Test Results -** Test reporting is continuous process. It may be both oral and written. It is important that defects and concerns be reported to the appropriate parties as early as possible, so that corrections can be made at the lowest possible cost.

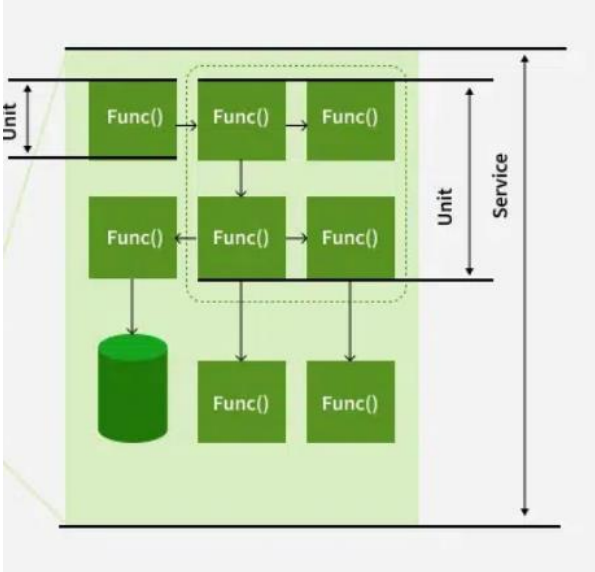
- **Step-9: The Software Installation -** Once test team has confirmed that software is prepared for production use, power to execute that software during production environment should be tested. This tests interface to operating software, related software, and operating procedures.

- **Step-10: Test Software Changes -** While this is often shown as Step 10, within context of performing maintenance after software is implemented, concept is additionally applicable to changes throughout implementation process. Whenever requirements changes, test plan must change, and impact of that change on software systems must be tested and evaluate.

- **Step-11: Evaluate Test Effectiveness -** Testing improvement can best be achieved by evaluating effectiveness of testing at top of every software test assignment. While this assessment is primarily performed by testers, it should involve developers, users of software, and quality assurance professionals if function exists within the IT organization.



Unit Testing :- A unit test is a small piece of code that checks if a specific function or method in is an application works correctly. It will work as the function inputs and verifying the outputs. These tests check that the code work as expected based on the logic the developer intended.

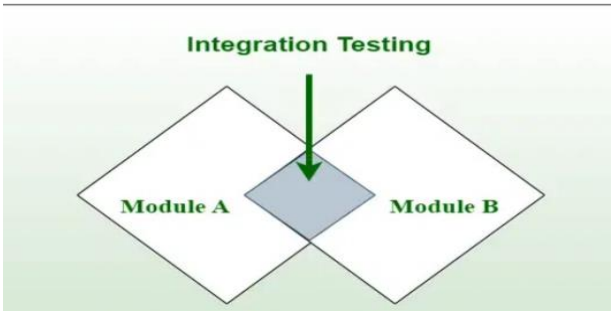


Unit testing strategies

To create effective unit tests, follow these basic techniques to ensure all scenarios are covered:

- **Logic checks**: Verify if the system performs correct calculations and follows the expected path with valid inputs. Check all possible paths through the code are tested.
- **Boundary checks**: Test how the system handles typical, edge case, and invalid inputs. For example, if an integer between 3 and 7 is expected, check how the system reacts to a 5 (normal), a 3 (edge case), and a 9 (invalid input).
- **Error handling**: Check the system properly handles errors. Does it prompt for a new input, or does it crash when something goes wrong?
- **Object-oriented checks**: If the code modifies objects, confirm that the object's state is correctly updated after running the code

Integration Testing:-



Integration Testing is a Software Testing Technique that focuses on verifying the interactions and data exchange between different components or modules of a Software Application. The goal of Integration Testing is to identify any problems or bugs that arise when different components are combined and interact with each other.

- It mainly tests interface between two software units or modules. It focuses on determining the correctness of the interface. Once all the modules have been unit-tested, integration testing is performed.
- Integration testing can be done by picking module by module. This can be done so that there is a proper sequence to be followed.
- Exposing the defects is the major focus of the integration testing and the time of interaction between the integrated units.

Why is Integration Testing Important?

Integration testing is important because it verifies that individual software modules or components work together correctly as a whole system. This ensures that the integrated software functions as intended and helps identify any compatibility or communication issues between different parts of the system. By detecting and resolving integration problems early, integration testing contributes to the overall reliability, performance, and quality of the software product.

How to Write Integration Tests?

Designing integration test cases is a key part of ensuring that the different components of your software work well together. Here's a simplified approach to designing these tests:

- **Identify the components to be tested**: Start by pinpointing which parts of your software need to be tested together. These are usually modules that interact or depend on each other.
- **Determine the test objectives**: Define what you want to achieve with the test. Are you testing if data flows correctly between modules? Or perhaps checking if the system behaves as expected when components interact?
- **Define the test data**: Decide what data you'll use to test the integration. Make sure the data represents real-world scenarios so that your tests are relevant and meaningful.
- **Design the test cases**: Plan out the specific steps for each test. Think about what actions the test will take and what results you expect.
- **Develop test scripts**: Write the code that will automate your tests. If your tests are manual, ensure the steps are clearly documented and easy to follow.
- **Set up the testing environment**: Make sure the environment where the tests will run mimics the real-world setup as closely as possible. This will give you more accurate results.
- **Execute the tests**: Run your tests, paying close attention to how the components interact and whether they perform as expected.
- **Evaluate the results**: Finally, review the test outcomes. Did the components work as intended? Were there any errors or unexpected behaviors?

Types of Integration Testing

1. **Big-Bang Integration Testing**

   It is the simplest integration testing approach, where all the modules are combined and the functionality is verified after the completion of individual module testing. In simple words, all the modules of the system are simply put together and tested.

- In debugging errors reported during Big Bang integration testing is very expensive to fix.
- Big-bang integration testing is a software testing approach in which all components or modules of a software application are combined and tested at once.

2. Bottom-Up Integration Testing

In bottom-up testing, each module at lower levels are tested with higher modules until all modules are tested. The primary purpose of this integration testing is that each subsystem tests the interfaces among various modules making up the subsystem

3. Top-Down Integration Testing

Top-down integration testing technique is used in order to simulate the behaviour of the lower-level modules that are not yet integrated. In this integration testing, testing takes place from top to bottom

4. Mixed Integration Testing

A mixed integration testing is also called sandwiched integration testing. A mixed integration testing follows a combination of top down and bottom-up testing approaches. In top-down approach, testing can start only after the top-level module have been coded and unit tested. In bottom-up approach, testing can start only after the bottom level modules are ready.

Unit Testing vs Integration Testing

S. No.	Unit Testing	Integration Testing
1.	In unit testing, each module of the software is tested separately.	In integration testing, all modules of the software are tested combined.
2.	In unit testing tester knows the internal design of the software.	Integration testing doesn't know the internal design of the software.
3.	Unit testing is performed first of all testing processes.	Integration testing is performed after unit testing and before system testing.
4.	Unit testing is white box testing.	Integration testing is black box testing.
5.	Unit testing is performed by the developer.	Integration testing is performed by the tester.

System Testing:-

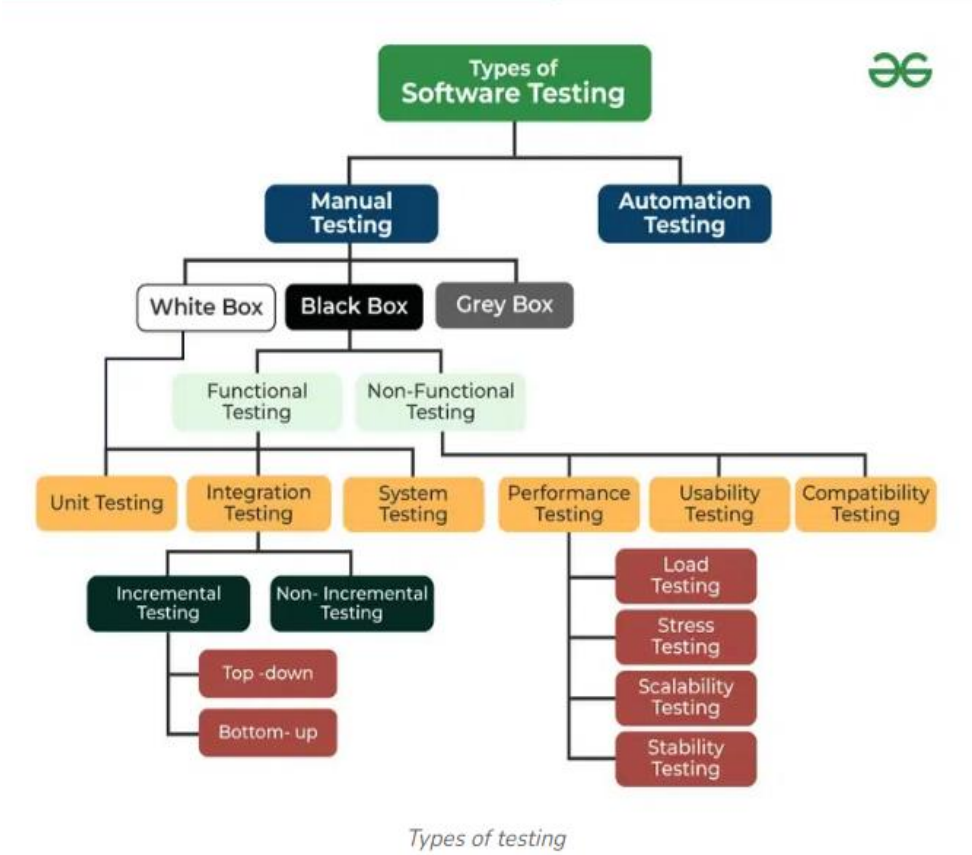
System Testing Process

System Testing is performed in the following steps:

- **Test Environment Setup:** Create testing environment for the better quality testing.
- **Create Test Case:** Generate test case for the testing process.
- **Create Test Data:** Generate the data that is to be tested.
- **Execute Test Case:** After the generation of the test case and the test data, test cases are executed.
- **Defect Reporting:** Defects in the system are detected.
- **Regression Testing:** It is carried out to test the side effects of the testing process.
- **Log Defects:** Defects are fixed in this step.
- **Retest:** If the test is not successful then again test is performed.

Types of System Testing

Here are the Types of System Testing are follows:



Types of testing

- **Functional Testing:** This checks if the system's features work as expected and meet the defined requirements.
- **Performance Testing:** This tests how the system performs under different conditions, like high traffic or heavy use, to ensure it can handle the expected load.
- **Security Testing:** This ensures the system's security measures protect sensitive data from unauthorized access or attacks.
- **Compatibility Testing:** This makes sure the system works well across different hardware, software, and network environments.
- **Usability Testing:** This evaluates how easy and user-friendly the system is, making sure it provides a good experience for users.
- **Regression Testing:** This ensures that any new code or features don't break or negatively affect the system's existing functionality.
- **Acceptance Testing:** This tests the system at a high level to make sure it meets customer expectations and requirements before release.

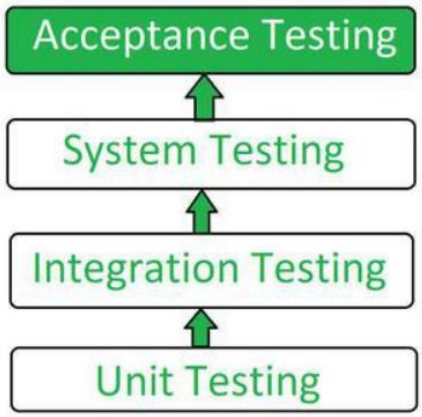
Tools used for System Testing

Here are the Tools used for System Testing are follows:

1. JMeter
2. Gallen Framework
3. HP Quality Center/ALM
4. IBM Rational Quality Manager
5. Microsoft Test Manager
6. Selenium
7. Appium
8. LoadRunner
9. Gatling

Acceptance Testing - Software Testing

Acceptance Testing is a formal testing according to user needs, requirements, and business processes conducted to determine whether a system satisfies the acceptance criteria or not and to enable the users, customers, or other authorized entities to determine whether to accept the system or not.

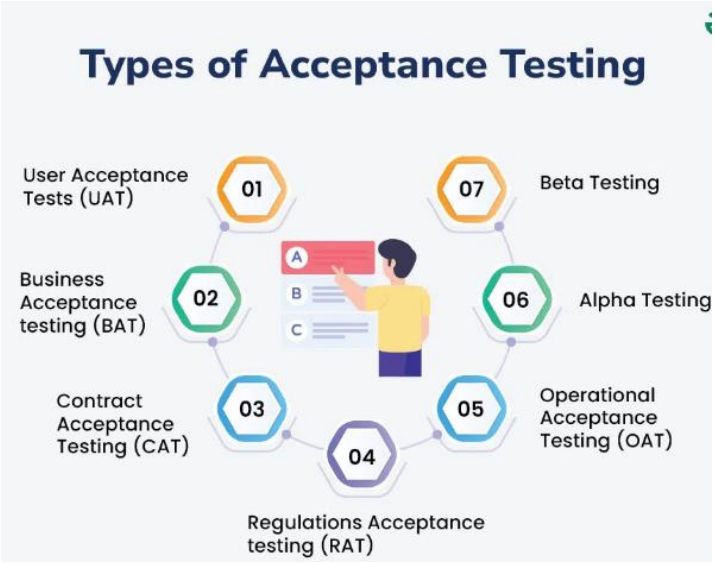


Flow of Acceptance Testing

Types of Acceptance Testing

Here are the Types of Acceptance Testing

1. User Acceptance Testing (UAT)
2. Business Acceptance Testing (BAT)
3. Contract Acceptance Testing (CAT)
4. Regulations Acceptance Testing (RAT)
5. Operational Acceptance Testing (OAT)
6. Alpha Testing
7. Beta Testing



1. User Acceptance Testing (UAT)

- User acceptance testing is used to determine whether the product is working for the user correctly.
- Specific requirements which are quite often used by the customers are primarily picked for testing purposes. This is also termed as End-User Testing.

2. Business Acceptance Testing (BAT)

- BAT is used to determine whether the product meets the business goals and purposes or not.
- BAT mainly focuses on business profits which are quite challenging due to the changing market conditions and new technologies, so the current implementation may have to being changed which results in extra budgets.

3. Contract Acceptance Testing (CAT)

- CAT is a contract that specifies that once the product goes live, within a predetermined period, the acceptance test must be performed, and it should pass all the acceptance use cases.

4. Regulations Acceptance Testing (RAT)

- RAT is used to determine whether the product violates the rules and regulations that are defined by the government of the country where it is being released.

5. Operational Acceptance Testing (OAT)

- OAT is used to determine the operational readiness of the product and is non-functional testing.

6. Alpha Testing

- Alpha testing is used to determine the product in the development testing environment by a specialized testers team usually called alpha testers.

7. Beta Testing

- Beta testing is used to assess the product by exposing it to the real end-users, typically called beta testers in their environment.
- Feedback is collected from the users and the defects are fixed. Also, this helps in enhancing the product to give a rich user experience.

Big Bang (Bing-Bang) Approach

Definition

In the **Big Bang approach**, **all modules are integrated at once** after unit testing, and the entire system is tested as a whole.

How it works

- Individual modules are developed and unit-tested separately
- All modules are combined in a single step
- System testing is performed after full integration

Advantages

- Simple and easy to implement
- Suitable for **small systems**
- No need for stubs or drivers

Disadvantages

- Difficult to locate errors
- High risk if defects appear late
- Not suitable for large or complex systems
- Debugging becomes complex

Example

A small calculator app where all functions (add, subtract, multiply, divide) are integrated together and tested at once.

Sandwich (Hybrid) Approach :- Definition

The **Sandwich approach** is a **hybrid integration testing strategy** that combines:

- **Top-down integration testing**
- **Bottom-up integration testing**

How it works

- Top-level modules are tested using **stubs**
- Bottom-level modules are tested using **drivers**
- Integration meets in the middle layer

Advantages

- Parallel testing is possible
- Faster defect detection
- Suitable for **large and complex systems**
- Reduces dependency on stubs/drivers

Disadvantages

- Planning is complex
- Requires more resources
- Middle-layer defects may be detected late

Example In a:- banking system:

- UI and business logic tested top-down
- Database and service layers tested bottom-up
- Integration converges at transaction-processing layer

1. Performance Testing

Definition

Performance testing evaluates how a system behaves under **various workloads** to ensure it meets **speed, responsiveness, stability, and scalability** requirements.

Purpose

- Measure response time
- Check system throughput
- Identify bottlenecks
- Ensure stability under expected load

Types

- Load testing
- Stress testing
- Spike testing
- Endurance (Soak) testing

Tools

JMeter, LoadRunner, Gatling

Example

Checking how fast an e-commerce website responds when 500 users browse products simultaneously.

2. Regression Testing

Definition

Regression testing ensures that **new changes (bug fixes, enhancements)** have **not broken existing functionality**.

Purpose

- Verify old features still work
- Detect side effects of code changes

When Performed

- After bug fixes
- After adding new features
- After configuration changes

Tools

Selenium, TestNG, JUnit

Example

After fixing a login bug, testing registration, password reset, and dashboard features again.

3. Smoke Testing

Definition

Smoke testing is a **preliminary testing** performed on a new build to check whether **critical functionalities work**.

Purpose

- Validate build stability
- Decide whether to proceed with detailed testing

Characteristics

- Shallow and wide
- Covers core features only
- Usually automated

Example

Checking if application launches, user can log in, and main menu loads.

4. Load Testing

Definition

Load testing is a type of **performance testing** that checks system behavior under **expected user load**.

Purpose

- Determine system capacity
- Measure response time under normal load
- Detect performance degradation

Tools

Apache JMeter, LoadRunner

Example

Testing a banking app with 1,000 concurrent users performing fund transfers.

Stages of STLC

STLC (Software Testing Life Cycle) defines the **sequence of activities performed during software testing** to ensure quality, reliability, and requirement compliance.

Stages of STLC

1 Requirement Analysis

- Understand **functional and non-functional requirements**
- Identify testable requirements
- Determine testing types (manual/automation)

Output: Requirement Traceability Matrix (RTM)

2 Test Planning

- Define **testing strategy and scope**
- Estimate effort, cost, and schedule
- Select tools and resources

Output: Test Plan document

3 Test Case Development

- Write **test cases and test scripts**
- Prepare test data
- Review and update test cases

Output: Test cases, test data

4 Test Environment Setup

- Prepare **hardware, software, and network**
- Validate environment readiness

Output: Test environment ready

5 Test Execution

- Execute test cases
- Log defects and track status
- Perform **retesting and regression testing**

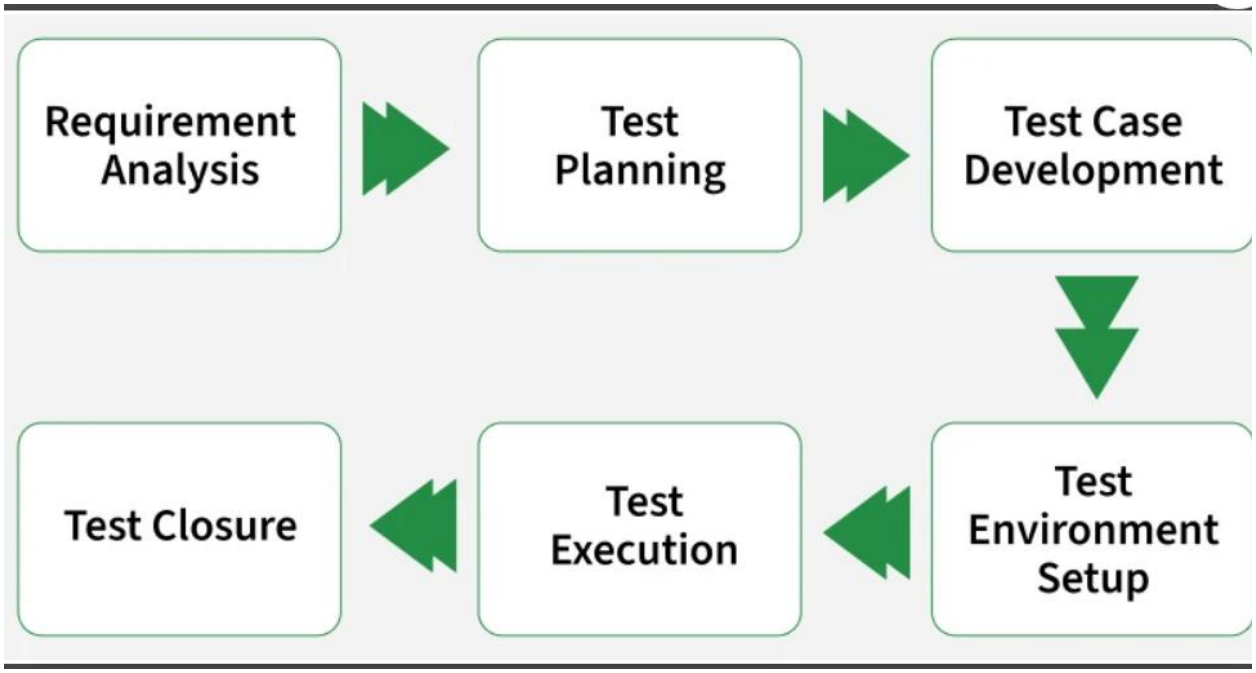
Output: Defect reports, test execution results

6 Test Cycle Closure

- Evaluate test completion criteria
- Prepare **test summary report**
- Analyze lessons learned

Output: Test closure report

The Software Testing Life Cycle (STLC) is a process that verifies whether the Software Quality meets the expectations or not. STLC is an important process that provides a simple approach to testing through the step-by-step process, which we are discussing here. Software Testing Life Cycle (STLC) is a fundamental part of the Software Development Life Cycle (SDLC).



1. Requirement Analysis

Requirement Analysis is the first phase where the QA/testing team understands what needs to be tested. The activities that take place during the Requirement Analysis stage include:

- Reviewing the software requirements document (SRD) and other related documents
- Interviewing stakeholders to gather additional information
- Identifying any ambiguities or inconsistencies in the requirements

- Identifying any missing or incomplete requirements
- Identifying any potential risks or issues that may impact the testing process

2. Test Planning

Test Planning is the most crucial phase where the overall test strategy and plan are created. The major activities carried out during the Test Planning phase include:

- Identifying the testing objectives and scope
- Developing a test strategy: selecting the testing methods and techniques that will be used
- Identifying the testing environment and resources needed
- Identifying the test cases that will be executed and the test data that will be used
- Estimating the time and cost required for testing
- Identifying the test deliverables and milestones
- Assigning roles and responsibilities to the testing team
- Reviewing and approving the test plan

3. Test Case Development

The Test Case Development phase testers design detailed test cases and prepare the necessary test data. The main activities performed during the Test Case Development phase include:

- Identifying the test cases that will be developed
- Writing test cases that are clear, concise and easy to understand
- Creating test data and test scenarios that will be used in the test cases
- Identifying the expected results for each test case
- Reviewing and validating the test cases
- Updating the requirement traceability matrix (RTM) to map requirements to test cases

4. Test Environment Setup

Test Environment Setup defines the hardware, software and network conditions under which testing will be executed. The activities involved in the Test Environment Setup phase include:

- Install and configure required software, tools and databases.
- Set up servers, browsers, operating systems and devices.
- Prepare access credentials and permissions.
- Validate the environment before test execution.

5. Test Execution

In Test Execution phase prepared test cases are executed in the defined environment. The activities performed during the Test Execution phase include:

- Run manual or automated test cases.
- Log defects with details like severity and priority.
- Retest fixed defects (defect retesting).
- Perform regression testing if required.
- Collect and analyze test results.
- Document and share test reports.

6. Test Closure

Test Closure is the final phase where testing activities are completed and documented. The final activities carried out during the Test Closure phase include:

- Prepare a Test Summary Report (test cases executed, pass/fail count, defects found/resolved).
- Ensure all defects are tracked and closed.
- Clean up the test environment.
- Archive test cases, data and reports.
- Conduct a retrospective for lessons learned.
- Share knowledge with stakeholders.